EKS ANYWHERE ON VMWARE vSPHERE

Backup & Disaster Recovery

etcd Backup · Stateful Workload Backup · Velero · Restore · Multi-Cluster DR

Instructor-Led Training | Platform SRE / Kubernetes Track

SESSION ROADMAP

What We'll Cover Today

1. **etcd Backup** — Snapshotting the Kubernetes control-plane's single source of truth
2. **Stateful Backup** — Protecting persistent volumes backed by the vSphere CSI driver
3. **Velero** — The backup/restore engine that ties cluster state and app data together
4. **Restore** — Recovering workloads, volumes and namespaces after loss or corruption
5. **Multi-Cluster DR** — Failing over an EKS Anywhere workload cluster to a second vSphere site

THE ON-PREM REALITY

Why DR Matters for EKS Anywhere

On managed EKS, AWS owns the control plane and its durability. On EKS Anywhere, the control plane runs on YOUR vSphere nodes — so etcd health, backup, and site resilience become the platform team's responsibility, not the cloud provider's.

Cloud EKS vs. EKS Anywhere

Control plane HA	AWS-managed	You architect it
etcd backups	AWS-managed	You schedule & own it
Underlying infra	AWS regions/AZs	Your vSphere hosts/datastores
Site-level failure	Multi-AZ automatic	Requires DR site design

Host / hardware failure

ESXi host, disk or network failure on a control-plane VM

Datastore corruption

Shared datastore issues affecting multiple node disks at once

Human / config error

Accidental delete of a namespace, CRD, or RBAC object

Full site outage

Power, vCenter, or network loss for an entire vSphere cluster

THE BIG PICTURE

DR Building Blocks, End to End

Every layer feeds the next: etcd protects control-plane state, CSI + Velero protect application data, and Velero's backups become the payload that a second cluster restores from during a DR event.

RPO

How much data you can afford to lose

RTO

How fast you must be back online

Runbook

The tested, repeatable recovery steps

Why etcd Is Critical

Everything the API server knows lives here

- Nodes, Pods, Deployments, Services, and every other object's desired & current state
- Secrets, ConfigMaps, and RBAC bindings — your entire security model
- CRDs and custom resources registered by add-ons and operators
- On EKS Anywhere the control plane (and etcd) runs on cluster nodes you manage — there is no AWS-managed etcd to fall back on

100%

of cluster state lives in etcd — lose it with no backup, and you rebuild the cluster from scratch

Bridge to DevSecOps

Think of etcd like your Git repo's object database — Kubernetes objects are the commits, etcd is where they actually live. Losing it is like losing .git with no remote.

Taking an etcd Snapshot

1. **SSH to a control-plane node**

   Any healthy control-plane VM in the vSphere-hosted cluster

2. **Run etcdctl snapshot save**

   Point at the local etcd endpoint with TLS certs

3. **Verify the snapshot**

   etcdctl snapshot status confirms hash, size and revision

4. **Ship it off-cluster**

   Copy to S3-compatible storage — never leave it only on the node

```
# Run on a control-plane node
$ export ETCDCTL_API=3
$ etcdctl snapshot save \
    /backup/etcd-snapshot-$(date +%F).db \
    --endpoints=https://127.0.0.1:2379 \
    --cacert=/etc/etcd/pki/ca.crt \
    --cert=/etc/etcd/pki/etcd.crt \
    --key=/etc/etcd/pki/etcd.key

# Verify the snapshot
$ etcdctl snapshot status \
    /backup/etcd-snapshot-*.db -w table

# Schedule via cron (every 6h)
0 */6 * * *  /usr/local/bin/etcd-backup.sh
```

Restoring etcd from a Snapshot

```
# 1. Stop kube-apiserver & etcd on ALL
#    control-plane nodes first
$ mv /etc/kubernetes/manifests/etcd.yaml \
    /tmp/

# 2. Restore into a fresh data dir
$ etcdctl snapshot restore \
    /backup/etcd-snapshot-2026-08-10.db \
    --data-dir=/var/lib/etcd-restored \
    --name=cp-node-1 \
    --initial-cluster=cp-node-1=https://...

# 3. Point etcd at the new data dir,
#    then restart the static pod
$ mv /tmp/etcd.yaml \
    /etc/kubernetes/manifests/
```

Do this on every control-plane node

A restore rebuilds a new etcd cluster identity. All control-plane nodes must restore from the same snapshot with consistent --initial-cluster settings, or the members won't agree.

What a restore recovers

- Namespaces, Deployments, Services
- Secrets, ConfigMaps, RBAC
- CRDs and custom resource objects

NOT the pod's data on disk — see stateful backup

Persistent Volumes: The Harder Problem

etcd tells you a PVC exists — it doesn't back up the bytes sitting on the vSphere datastore behind it. On EKS Anywhere, PersistentVolumes are typically provisioned by the vSphere CSI driver, so a real DR story needs volume-level snapshots, not just API-object snapshots.

etcd Backup

- Captures Kubernetes API objects
- PVC spec, StorageClass, claims
- Fast, small, cluster-wide

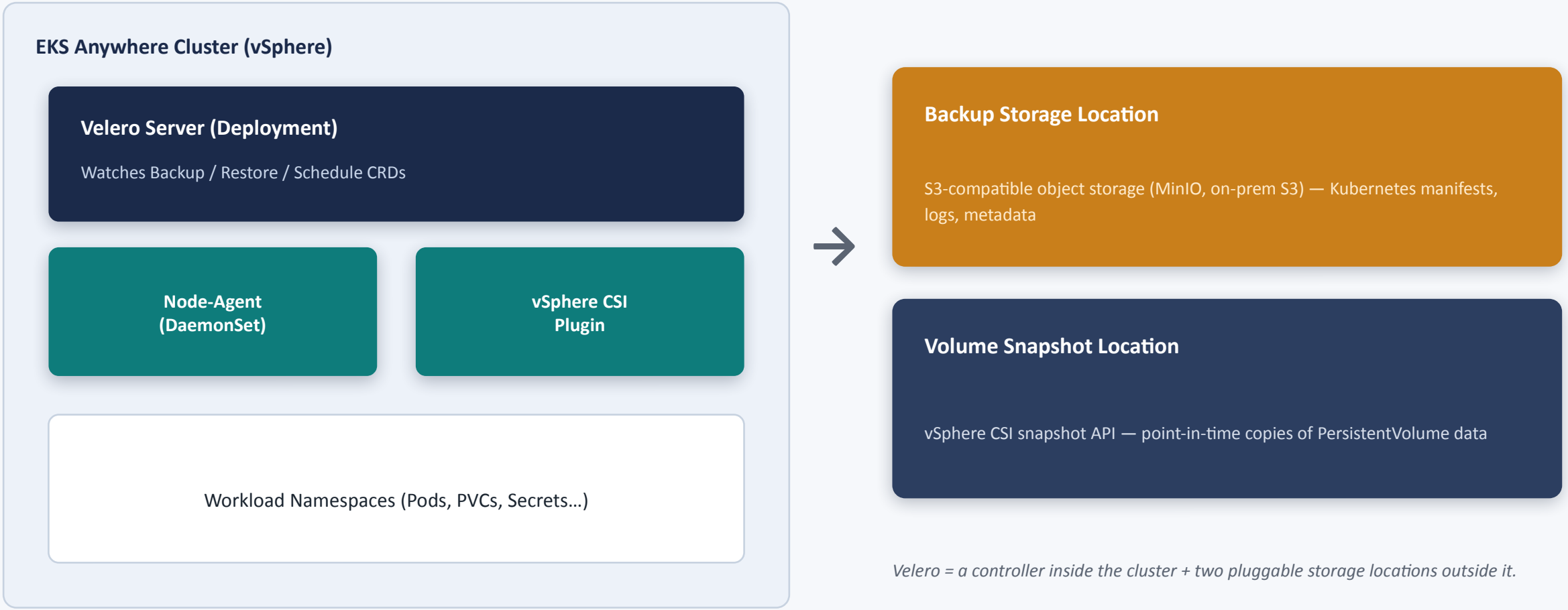
Volume Backup

- Captures actual data blocks
- Needs vSphere CSI snapshot support
- Larger, app-consistency matters

Together = DR-ready

- Object + data both protected
- Velero orchestrates both
- Restorable to a new cluster

Velero Architecture Overview



Velero = a controller inside the cluster + two pluggable storage locations outside it.

Installing & Configuring Velero

```
# Install Velero with vSphere plugins
$ velero install \
  --provider community.velero.io/aws \
  --plugins velero/velero-plugin-for-aws,\
    vsphereveleroplugin/velero-plugin-\
    for-vsphere \
  --bucket eksa-dr-backups \
  --secret-file ./credentials-minio \
  --backup-location-config \
    region=minio,s3Url=https://minio.\
    dr.local,s3ForcePathStyle="true" \
  --use-node-agent \
  --snapshot-location-config \
    provider=vsphere
```

What to configure once

- **BackupStorageLocation**

  Where manifests & metadata land (S3-compatible bucket)

- **VolumeSnapshotLocation**

  vSphere CSI as the volume-snapshot provider

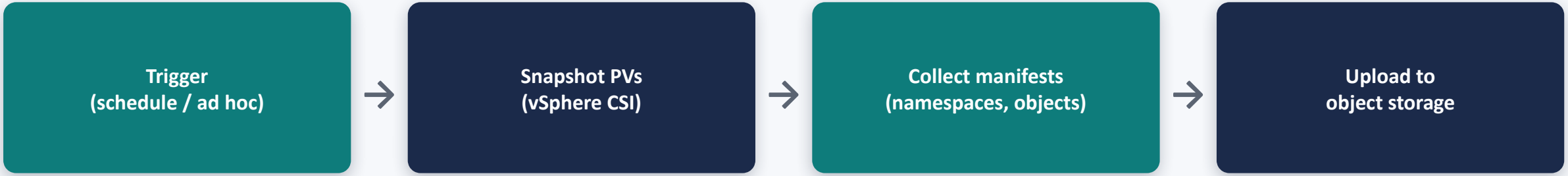
- **Node-Agent DaemonSet**

  Enables file-system backup/restore (fs-backup)

- **Credentials Secret**

  Access keys for the object storage bucket

The Velero Backup Workflow




```
# Recurring nightly backup, 30-day TTL
$ velero schedule create nightly-sms \
    --schedule="0 2 * * *" \
    --include-namespaces student-mgmt \
    --ttl 720h0m0s

# One-off backup before a risky change
$ velero backup create pre-upgrade-bkp \
    --include-namespaces student-mgmt
```

Fine-tune with hooks & selectors

- Pre-hooks: flush a database before snapshotting its PV
- Post-hooks: run validation after restore completes
- Label / namespace selectors to scope what's backed up
- Resource filters to exclude noisy or regenerable objects

Restoring with Velero

```
# List available backups
$ velero backup get

# Restore into the same cluster
$ velero restore create \
    --from-backup nightly-sms-20260817

# Restore into a DIFFERENT cluster,
# remapping the namespace
$ velero restore create \
    --from-backup nightly-sms-20260817 \
    --namespace-mappings \
      student-mgmt:student-mgmt-dr

# Watch progress
$ velero restore describe <name>
```

Things to check before you restore

- **StorageClass exists**

  Target cluster must have a matching vSphere CSI StorageClass

- **Namespace conflicts**

  Use --namespace-mappings to avoid overwriting live data

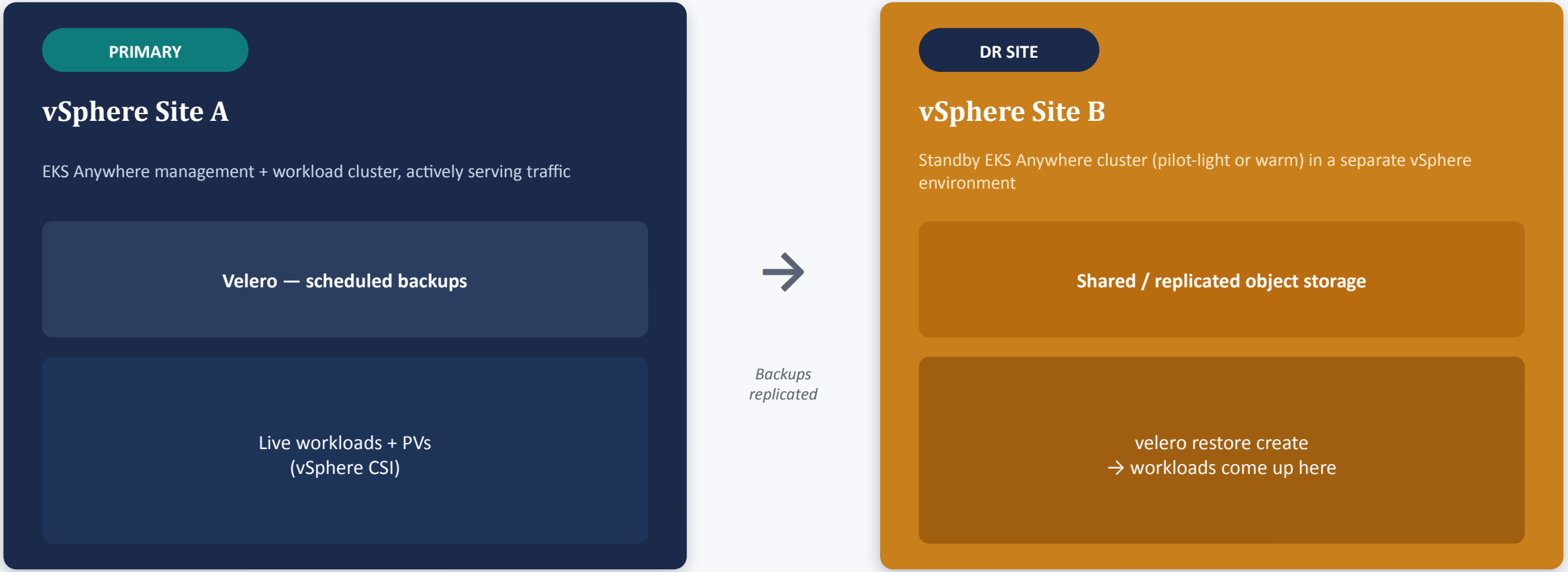
- **CRDs installed**

  Operators / CRDs referenced by backed-up objects must exist first

- **Restore order**

  Velero restores in dependency order, but verify PVCs bind cleanly

Two-Site DR Architecture



Object storage (S3-compatible) sits outside both vSphere sites, or is replicated between them, so a Site A loss doesn't take backups down with it.

Failover Process & RTO / RPO

1. Declare a disaster and freeze writes at the primary (if reachable)
2. Stand up / scale the DR-site EKS Anywhere workload cluster
3. Point Velero at the shared BackupStorageLocation and run velero restore
4. Validate workloads, PVC binding, and app health checks
5. Cut over DNS / load balancer / GitOps target to the DR cluster

RPO

Set by backup schedule interval

e.g. nightly = up to 24h data loss

RTO

Set by cluster stand-up + restore + cutover time

Pilot-light DR cuts RTO vs. cold start

BRINGING IT TOGETHER

DR Best-Practices Checklist

✓ Automate everything

Cron/CronJob etcd snapshots + Velero schedules — never rely on someone remembering

✓ Test restores, not just backups

A backup you've never restored is a guess, not a plan

✓ Store off-site / off-cluster

Object storage independent of the primary vSphere datastore and site

✓ Encrypt backups at rest & in transit

etcd snapshots and Velero buckets both contain Secrets

✓ Version & retain with a TTL policy

Keep enough history to recover from a slow-discovered corruption

✓ Document and rehearse the runbook

Time a real failover in a game-day so the RTO number is trustworthy

KEY TAKEAWAYS

Five Pillars of EKS Anywhere DR

1	**etcd Backup**	Protects cluster state — you own it on EKS Anywhere
2	**Stateful Backup**	vSphere CSI snapshots protect the actual data
3	**Velero**	The engine that automates and orchestrates both
4	**Restore**	Rehearsed, tested recovery — into the same or a new cluster
5	**Multi-Cluster DR**	A second vSphere site turns backups into real resilience

Thank you · Questions & Discussion